INSTITUTO FEDERAL
Paraná
Campus Pinhais

INSTITUTO FEDERAL DO PARANÁ

BEATRIZ REGINA NAHAS
JOÃO PEDRO DOS SANTOS HENRIQUE PLINTA

MANUAL 1 – EBPF PARA OBSERVABILIDADE

PINHAIS
2026

BEATRIZ REGINA NAHAS
JOÃO PEDRO DOS SANTOS HENRIQUE PLINTA

MANUAL 1 – EBPF PARA OBSERVABILIDADE

Manual produzido no âmbito do projeto de extensão do minicurso Introdução ao eBPF, apresentado à disciplina de Práticas de Extensão V, do curso de Bacharelado em Ciência da Computação do Instituto Federal do Paraná, Campus Pinhais, sob a orientação do professor Guilherme Werneck de Oliveira.

Orientador: Prof. Guilherme Werneck de Oliveira

PINHAIS
2026

SUMÁRIO

1 INTRODUÇÃO **3**
1.1 O QUE É EBPF 3
1.2 ARQUITETURA E FUNCIONAMENTO 4
1.3 EBPF APLICADO À OBSERVABILIDADE 5
2 PRÉ-REQUISITOS **7**
2.1 CONHECIMENTOS NECESSÁRIOS 7
2.2 FERRAMENTAS NECESSÁRIAS 7
2.3 INSTALAÇÃO E CONFIGURAÇÃO DO AMBIENTE 8
2.4 PROBLEMAS COMUNS 9
3 FUNDAMENTOS DE OBSERVABILIDADE **11**
3.1 O QUE É OBSERVABILIDADE 11
3.2 LOGS 11
3.3 MÉTRICAS 12
3.4 TRACES 12
3.5 EVENTOS 13
4 EBPF PARA MONITORAMENTO **14**
4.1 RASTREAMENTO DE PROCESSOS E CHAMADAS DE SISTEMA 14
4.2 MONITORAMENTO DE CPU E MEMÓRIA 15
4.3 MONITORAMENTO DE DISCO E I/O 15
4.4 MONITORAMENTO DE APLICAÇÕES 16
5 FERRAMENTAS DE OBSERVABILIDADE **17**
5.1 BCC 17
5.2 BPFTRACE 18
5.3 LIBBPF 19
5.4 EBPF EXPORTERS 21
6 INSTRUMENTAÇÃO COM EBPF **22**
6.1 KPROBES 22
6.2 KRETPROBES 23
6.3 TRACEPOINTS 23
6.4 UPROBES 24
6.5 USDT 24
6.6 PERF EVENTS 25
7 COLETA E VISUALIZAÇÃO DE DADOS **26**
7.1 EBPF E PROMETHEUS 26
7.2 EBPF E GRAFANA 27
7.3 EXPORTAÇÃO DE MÉTRICAS 27
7.4 COLETA DE EVENTOS 29
7.5 DASHBOARDS DE OBSERVABILIDADE 29
8 EXEMPLOS PRÁTICOS **31**
8.1 MONITORAMENTO DE PROCESSOS 31
8.2 ANÁLISE DE CHAMADAS DE SISTEMA 31
8.3 RASTREAMENTO DE UMA APLICAÇÃO 32
9 TROUBLESHOOTING **34**
9.1 IMPACTO DE PERFORMANCE 34
9.2 LIMITAÇÕES 34
10 CONCLUSÃO **36**

1 INTRODUÇÃO

Este manual apresenta o uso do eBPF (*extended Berkeley Packet Filter*) para observabilidade em sistemas Linux. O texto é dirigido a quem já usa Linux no dia a dia, conhece o terminal e ferramentas como `top`, `ss` e `iostat`, mas ainda não escreveu ou executou programas eBPF. O objetivo é sair de `/proc` e das ferramentas de *polling* e chegar a coletar métricas diretamente do kernel, entendendo o que cada peça faz e quanto ela custa.

1.1 O QUE É EBPF

O eBPF é uma tecnologia do kernel Linux que permite carregar e executar pequenos programas dentro do próprio kernel, sem alterar seu código-fonte e sem carregar módulos. Os programas são escritos em C restrito (ou em Rust), compilados para um *bytecode* próprio e executados por uma máquina virtual embutida no kernel (GREGG, 2019; Linux Kernel Documentation, 2024). Cada programa fica associado a um evento, como a chamada de uma função do kernel, a chegada de um pacote ou a execução de uma chamada de sistema, e é executado toda vez que esse evento ocorre.

A tecnologia vem do BPF clássico, proposto em 1993 por McCanne e Jacobson para filtrar pacotes dentro do kernel, em vez de copiar todos eles para o espaço do usuário e descartar a maioria depois (MCCANNE; JACOBSON, 1993). Esse filtro era uma máquina virtual minúscula, com dois registradores e um conjunto reduzido de instruções, e serviu de motor para ferramentas como o `tcpdump`. Em 2014, o kernel 3.18 incorporou uma reformulação completa, o eBPF, com onze registradores de 64 bits, mapas para guardar estado, um verificador estático e compilação JIT (GREGG, 2019). Com isso, o BPF deixou de ser um filtro de pacotes e passou a ser uma plataforma para estender o kernel em tempo de execução, sem reiniciar a máquina.

Diferente de um módulo de kernel, um programa eBPF não pode travar o sistema: ele só é carregado depois de aprovado pelo verificador, que rejeita código que possa não terminar ou acessar memória inválida (Linux Kernel Documentation, 2024). É essa garantia que permite executar código escrito por terceiros dentro do kernel em ambientes de produção. Sobre essa base foram construídas ferramentas de uso cotidiano, como o `bpftrace` e o BCC para diagnóstico, o Cilium para rede em Kubernetes e o Falco para detecção de comportamento suspeito.

A motivação, para a observabilidade, aparece ao comparar com as ferramentas tradicionais. Sysstat e Prometheus, por exemplo, leem arquivos de `/proc` em intervalos fixos: a cada leitura o processo cruza a fronteira entre espaço do usuário e kernel, recebe os contadores em texto e faz o *parsing*. Sob alta carga, esse ciclo disputa CPU com as aplicações que deveria apenas observar (CASSAGNES *et al.*, 2020). Com eBPF, a agregação acontece no kernel, no momento do evento, e o espaço do usuário só lê o resultado quando precisa.

1.2 ARQUITETURA E FUNCIONAMENTO

Um programa eBPF passa por um ciclo fixo, do código-fonte à execução:

1. o código em C é compilado com o `clang` para *bytecode* eBPF, gerando um objeto ELF;
2. um programa em espaço do usuário, normalmente baseado na `libbpf`, carrega o objeto e o envia ao kernel pela chamada de sistema `bpf()`;
3. o **verificador** analisa o *bytecode* antes de qualquer execução e rejeita o programa se ele puder não terminar, acessar memória fora dos limites ou usar ponteiros sem checagem de nulo (Linux Kernel Documentation, 2024);
4. o compilador **JIT** traduz o *bytecode* aprovado para instruções nativas do processador, de modo que o programa roda sem camada de interpretação (GREGG, 2019);
5. o programa é anexado a um ponto de ancoragem (*hook*) e passa a ser executado a cada ocorrência do evento;
6. os resultados são gravados em **mapas eBPF**, estruturas de dados compartilhadas entre o kernel e o espaço do usuário.

A máquina virtual possui onze registradores de 64 bits (`r0` a `r10`) e uma pilha de apenas 512 bytes por programa. Dados que precisam durar mais que uma execução, ou chegar ao espaço do usuário, vão para os mapas, que existem em vários tipos: tabelas *hash*, *arrays*, versões por CPU (que evitam disputa entre núcleos) e o *ring buffer*, usado para enviar eventos ao espaço do usuário (GREGG, 2019). O programa também não pode chamar funções arbitrárias do kernel, apenas as *helper functions* liberadas para o seu tipo, como as que consultam o PID do processo atual, leem o relógio ou acessam um mapa.

O Quadro 1 resume os principais pontos de ancoragem para observabilidade. A escolha do *hook* define o que o programa enxerga e qual o custo: quanto mais frequente o evento, mais vezes o programa roda.

Para acessar estruturas internas do kernel sem depender da versão em uso, o eBPF moderno usa o modelo CO-RE (*Compile Once, Run Everywhere*). O kernel expõe a descrição de seus tipos no formato BTF, em `/sys/kernel/btf/vmlinux`, e a `libbpf` ajusta, no momento do carregamento, os deslocamentos dos campos acessados pelo programa (NAKRYIKO, 2020). Assim, um mesmo binário compilado uma vez funciona em kernels diferentes, sem instalar compilador nem cabeçalhos na máquina de destino.

Quadro 1 – Principais pontos de ancoragem do eBPF

Hook	Onde se conecta	Uso típico
kprobe, kretprobe	entrada e saída de uma função do kernel	contar bytes em `tcp_sendmsg`
tracepoint	pontos estáveis definidos pelo kernel	chamadas de sistema, como `sys_enter_openat`
uprobe	função de um binário em espaço do usuário	latência de uma função da aplicação
perf event	eventos de hardware e software, com amostragem	perfil de uso de CPU
TC e XDP	tráfego de rede, no TC ou direto no driver	medir e filtrar pacotes

Fonte: Elaborado pelos autores.

1.3 EBPF APLICADO À OBSERVABILIDADE

Na observabilidade, o eBPF permite capturar eventos individuais com granularidade de microssegundos, como a latência de uma única conexão TCP ou a duração de uma chamada de sistema, sem instrumentar a aplicação e sem recompilar o kernel (GREGG, 2019). Isso responde a perguntas que *polling* não responde bem: quais processos abrem mais arquivos agora, qual a latência de cada `read` em disco, quantos bytes passam por cada porta TCP. Como a coleta é orientada a eventos, ela também enxerga ocorrências curtas que desaparecem entre duas amostras de uma ferramenta de *polling* (RESEARCH, 2024).

Há dois modos de uso. O primeiro é interativo: ferramentas como `bpftrace` e BCC permitem escrever ou rodar *scripts* curtos para investigar um problema em andamento. O segundo é contínuo: um programa próprio, em C com `libbpf`, fica carregado, agrega dados em mapas e os expõe a um painel ou ao Prometheus. A partir do capítulo 5, os dois modos são tratados.

No experimento do trabalho de conclusão de curso que originou este material (ALVES; PLINTA, 2026), o coletor eBPF teve o menor tempo de resposta médio nos quatro volumes de carga testados (de 100.000 a 2.000.000 de mensagens, com 30 execuções por configuração): entre 0,49 e 0,54 ms, contra 0,57 a 0,62 ms do Sysstat e do Prometheus, uma vantagem de 8% a 15% com intervalos de confiança de 95% sem sobreposição. Já o consumo de CPU do próprio coletor ficou inconclusivo: o eBPF teve a menor média apenas no menor volume, e mesmo ali os intervalos das outras ferramentas incluem o valor dele.

Isso não significa que o eBPF substitua as demais ferramentas. Ele exige um kernel recente (de preferência 5.8 ou superior), conhecimento mais especializado e cuidado com o desempenho: um programa anexado a uma função chamada milhões de vezes por segundo, com lógica pesada, pode degradar o host inteiro (Linux Kernel Documentation, 2024). Os capítulos seguintes mostram como montar o ambiente, quais ferramentas usar e como levar as métricas

coletadas até um painel.

2 PRÉ-REQUISITOS

Este capítulo lista o que é necessário saber e instalar antes de seguir o manual, e como verificar se a máquina está pronta. Os comandos foram pensados para Ubuntu, a distribuição usada no minicurso. Em outras distribuições os nomes dos pacotes mudam, mas as ferramentas e os requisitos de kernel são os mesmos.

2.1 CONHECIMENTOS NECESSÁRIOS

Não é preciso saber programar para o kernel, mas alguns conhecimentos são assumidos:

- uso confortável do terminal Linux, de `sudo` e do gerenciador de pacotes da distribuição;
- noção de processo, chamada de sistema (*syscall*) e da separação entre espaço do usuário e kernel;
- leitura básica de C, suficiente para acompanhar os exemplos de *struct*, ponteiros e funções;
- noções de rede (TCP, portas) e de métricas de sistema (CPU, memória, I/O), úteis nos exemplos práticos;
- familiaridade com `/proc`, por exemplo `/proc/stat` e `/proc/net/dev`. Não é obrigatória, mas ajuda a entender por que o eBPF é uma alternativa.

Quem já escreveu um módulo de kernel ou usou `perf` e `strace` vai reconhecer vários conceitos, mas nada disso é exigido.

2.2 FERRAMENTAS NECESSÁRIAS

- **Kernel Linux 5.4 ou superior**, com BTF habilitado (`CONFIG_DEBUG_INFO_BTF=y`). Para os recursos mais novos, como o *ring buffer*, prefira o 5.8 ou superior. A maioria das instalações Ubuntu 20.04 ou mais recentes já atende.
- **Permissão de superusuário** (`root`) para carregar programas no kernel. A partir do kernel 5.8, as capacidades `CAP_BPF` e `CAP_PERFMON` permitem fazer isso sem `root` completo (Linux Kernel Documentation, 2024).
- **clang e LLVM**, que compilam o C para *bytecode* eBPF.
- **libbpf e libelf**, para carregar e gerenciar os programas.

- **bpftool**, para inspecionar programas e mapas carregados e gerar o cabeçalho `vmlinux.h`.
- **bpftrace** e **libbpf-tools**, para consultas rápidas e ferramentas prontas. O **BCC** é opcional (capítulo 5).
- **Docker**, opcional, para os exemplos com contêineres.

Além do BTF, o kernel precisa ter o suporte a eBPF e a instrumentação compilados. As opções `CONFIG_BPF`, `CONFIG_BPF_SYSCALL`, `CONFIG_BPF_JIT`, `CONFIG_BPF_EVENTS`, `CONFIG_KPROBES` e `CONFIG_UPROBES` devem estar com valor `y`, o que é o padrão nas distribuições comuns. Três sistemas de arquivos também são usados: `/sys/fs/bpf` (para fixar objetos), `/sys/kernel/tracing` e `/sys/kernel/debug`. Normalmente já vêm montados.

2.3 INSTALAÇÃO E CONFIGURAÇÃO DO AMBIENTE

Primeiro, verifique se o kernel atende aos requisitos:

```
uname -r
ls /sys/kernel/btf/vmlinux
grep CONFIG_DEBUG_INFO_BTF /boot/config-$(uname -r)
```

O segundo comando deve listar o arquivo sem erro, e o terceiro deve retornar a linha `CONFIG_DEBUG_INFO_BTF=y`. Se o arquivo não existir, o kernel não expõe BTF e os programas CO-RE deste manual não vão carregar. Para conferir as demais opções de uma vez:

```
grep -E '^CONFIG_(BPF|BPF_SYSCALL|BPF_JIT|BPF_EVENTS|KPROBES|UPROBES)='
   ↪ \
    /boot/config-$(uname -r)
```

Em seguida, instale as dependências. A lista abaixo foi validada em uma instalação limpa do Ubuntu 26.04:

```
sudo apt update
sudo apt install -y clang llvm libbpf-dev libelf-dev \
    linux-tools-$(uname -r) \
    linux-headers-$(uname -r) \
    bpftrace libbpf-tools python3 python3-pip
```

O pacote `libbpf-tools` instala as ferramentas prontas do capítulo 4, com o sufixo `-libbpf` (por exemplo, `execsnoop-libbpf`). O BCC, instalado pelo pacote `bpfcc-tools` e com o sufixo `-bpfcc`, é opcional e pode falhar em kernels muito recentes, como descrito na seção 2.4.

O pacote `linux-tools` traz o `bpftool`. Com ele, gere o cabeçalho com todos os tipos do kernel em uso, que os programas em C incluirão no lugar dos cabeçalhos tradicionais:

```
bpftool btf dump file /sys/kernel/btf/vmlinux format c > vmlinux.h
```

O arquivo gerado tem mais de cem mil linhas e deve ser regenerado quando o kernel for atualizado. Em seguida, confira se o `bpftool` enxerga o sistema com `sudo bpftool prog show`, que lista os programas eBPF já carregados (o sistema costuma ter alguns, do `systemd`, por exemplo).

Para confirmar que tudo funciona, rode um programa mínimo de `bpftrace` que conta as chamadas `openat` por processo. Deixe-o rodando alguns segundos e encerre com `Ctrl+C`, que imprime o mapa acumulado:

```
sudo bpftrace -e 'tracepoint:syscalls:sys_enter_openat { @[comm] = count
    ↪ (); }'
```

Ao encerrar, o `bpftrace` imprime o mapa, ordenado da menor para a maior contagem. Este é um trecho de uma execução real de 3 segundos:

```
@[pool-870]: 8
@[blocking-587]: 8
@[systemd-journal]: 14
@[blocking-588]: 17
@[blocking-585]: 25
@[bwrap]: 33
```

Os nomes como `blocking-588` e `pool-870` são nomes de *threads*, que o campo `comm` também registra. Se a saída listar nomes de processos com suas contagens, o ambiente está pronto.

2.4 PROBLEMAS COMUNS

- `Operation not permitted`: o comando precisa de `root`. Repita com `sudo`. Sem ele, até o `bpftool prog show` falha. O parâmetro `kernel.unprivileged_bpf_disabled`, que pode ser consultado com `sysctl`, define se usuários comuns podem carregar programas, e em muitas distribuições vem desabilitado.
- `/sys/kernel/btf/vmlinux` inexistente: o kernel não foi compilado com BTF. Atualize para um kernel mais novo da distribuição.
- Cabeçalhos ou `bpftool` de outra versão: depois de atualizar o kernel, reinstale `linux-tools` e `linux-headers` para a versão de `uname -r`.
- `Failed to compile BPF module` em uma ferramenta `-bpfcc`: o BCC compila o programa na hora, contra os cabeçalhos do kernel, e versões mais antigas do BCC não acom-

panham kernels muito recentes. Use a versão `-libbpf` da mesma ferramenta, ou um *script* de `bpftrace`.

- Erro do `bpftrace` sobre o *tracepoint*: confira o kernel e se o `/sys/kernel/tracing` está montado.
- Máquinas virtuais e WSL: nem todas as imagens trazem um kernel com BTF e *tracepoints*. Em caso de dúvida, rode as verificações acima antes de seguir.

3 FUNDAMENTOS DE OBSERVABILIDADE

Antes de usar o eBPF para coletar dados, é preciso saber o que se quer observar e em que forma esses dados aparecem. Este capítulo apresenta o conceito de observabilidade e os tipos de dado que ela usa, com destaque para onde o eBPF se encaixa em cada um.

3.1 O QUE É OBSERVABILIDADE

Observabilidade é a capacidade de entender o estado interno de um sistema a partir dos dados que ele produz, sem precisar alterá-lo para cada nova pergunta (MAJORS; FONG-JONES; MIRANDA, 2022). A diferença para o monitoramento tradicional está no tipo de pergunta: o monitoramento avisa que algo conhecido saiu do normal (a CPU passou de 90%), enquanto a observabilidade deve permitir investigar problemas que ninguém previu (por que as requisições na porta 8080 ficam lentas só às terças?). Na prática, as duas se complementam: alertas dizem que existe um problema, e dados detalhados ajudam a descobrir a causa.

Os dados costumam ser agrupados em três categorias, chamadas de pilares: *logs*, métricas e *traces*. Este manual trata ainda de uma quarta forma, os eventos, por ser a matéria-prima natural do eBPF. O Quadro 2 resume as quatro e o papel do eBPF em cada uma.

Quadro 2 – Tipos de dado de observabilidade e o papel do eBPF

Tipo	**O que registra**	**Exemplo**	**Papel do eBPF**
Logs	texto com marca de tempo	`journalctl`	gera registros sobre o que o kernel viu
Métricas	valores numéricos agregados	`vmstat 1`	agrega no kernel, a cada evento
Traces	caminho de uma requisição	*spans* com ID de *trace*	cobre apenas a parte visível ao kernel
Eventos	ocorrência individual	`execve`, pacote recebido	captura direta, no *hook*

Fonte: Elaborado pelos autores.

3.2 LOGS

Logs são registros de texto, com marca de tempo, que descrevem algo que aconteceu: uma requisição atendida, um erro, uma falha de autenticação. Em Linux, aparecem em arquivos de `/var/log` e no `journald`, que se consulta com `journalctl`. São ricos em contexto, mas dependem de o desenvolvedor ter previsto aquela mensagem, e o volume cresce rápido, o que

encarece armazenamento e busca. Um *log* que ninguém escreveu não pode ser consultado depois.

O eBPF não substitui os *logs* da aplicação, mas permite gerar registros sobre o que o kernel viu, como processos executados, arquivos abertos ou conexões estabelecidas, sem mexer no código observado. É útil justamente quando a aplicação não registra o que se precisa saber, ou quando o código-fonte não está disponível.

3.3 MÉTRICAS

Métricas são valores numéricos agregados ao longo do tempo, como taxa de requisições, uso de memória ou bytes transmitidos por segundo. São baratas de armazenar e boas para painéis e alertas. No modelo do Prometheus, por exemplo, uma métrica pode ser um contador (que só cresce, como o total de bytes), um *gauge* (que sobe e desce, como a memória em uso) ou um histograma (que distribui valores em faixas, como latências) (TURNBULL, 2018). O mesmo modelo usa um servidor que busca periodicamente, por HTTP, as métricas expostas pelos *exporters*.

Duas referências ajudam a decidir quais métricas coletar. Os quatro sinais dourados do livro de engenharia de confiabilidade do Google são latência, tráfego, erros e saturação (BEYER *et al.*, 2016). O método USE, de Brendan Gregg, propõe examinar, para cada recurso (CPU, memória, disco, rede), a utilização, a saturação e os erros (GREGG, 2020). As duas ideias aparecem nos exemplos práticos deste manual.

A limitação das métricas é a resolução. Uma métrica coletada a cada segundo esconde um pico de 200 ms que começou e terminou entre duas amostras (RESEARCH, 2024). Também há o custo de cardinalidade: cada combinação de rótulos (por exemplo, uma métrica por porta e por processo) vira uma série temporal a mais para armazenar. Com eBPF, a agregação ocorre no kernel a cada evento, e o espaço do usuário apenas lê o contador já consolidado (GREGG, 2019).

3.4 TRACES

Traces acompanham o caminho de uma requisição por vários componentes, registrando quanto tempo ela passou em cada etapa. Cada etapa é um *span*, e todos os *spans* de uma mesma requisição compartilham um identificador, que precisa ser repassado de um serviço para o outro (propagação de contexto). São os mais úteis em arquiteturas de microsserviços, onde uma única chamada atravessa vários serviços, e mostram onde o tempo está sendo gasto.

Normalmente exigem instrumentação na aplicação, em geral por meio de bibliotecas como o OpenTelemetry. O eBPF cobre a parte que fica visível ao kernel, como o tempo entre

o envio e a resposta de uma conexão TCP, ou o tempo gasto em uma chamada de sistema. Ele não vê o que acontece dentro da lógica da aplicação, então complementa os *traces* em vez de substituí-los.

3.5 EVENTOS

Evento é a ocorrência individual de algo no sistema: um `execve`, o recebimento de um pacote, uma escrita em disco. Logs, métricas e *traces* podem ser vistos como formas de resumir eventos. O eBPF opera diretamente nesse nível: um programa anexado a um *hook* é executado a cada evento, e pode registrá-lo inteiro, enviando-o ao espaço do usuário, ou apenas atualizar um contador em um mapa (GREGG, 2019).

Essa escolha é o principal ajuste de projeto de uma coleta com eBPF. Registrar cada evento dá o máximo de detalhe, mas custa mais e gera volume. Agregar no kernel é mais barato e escala melhor, mas perde o detalhe individual. É também a razão de o eBPF alcançar a granularidade de microssegundos, e de o custo precisar de atenção: quanto mais frequente o evento, mais vezes o programa roda.

4 EBPF PARA MONITORAMENTO

Este capítulo mostra o que o eBPF permite monitorar em um host Linux: processos e chamadas de sistema, CPU e memória, disco e aplicações. Cada seção usa o roteiro do método USE, que examina utilização, saturação e erros de cada recurso (GREGG, 2020), e apresenta comandos prontos para executar. Os exemplos usam dois tipos de ferramenta: as ferramentas prontas do pacote `libbpf-tools`, que no Ubuntu têm o sufixo `-libbpf` (por exemplo, `execsnoop-libbpf`), e *scripts* de `bpftrace`. As ferramentas `-libbpf` usam CO-RE e não compilam nada na hora da execução, o que as torna mais confiáveis que as equivalentes do BCC (seção 5.1). Todos os comandos foram testados em Ubuntu 26.04 com kernel 7.0 e exigem `root`.

4.1 RASTREAMENTO DE PROCESSOS E CHAMADAS DE SISTEMA

Chamadas de sistema são o ponto onde toda aplicação pede algo ao kernel: abrir um arquivo, criar um processo, enviar dados. Observá-las mostra o que um programa realmente faz, sem ler seu código. A ferramenta clássica para isso é o `strace`, que usa `ptrace` e interrompe o processo a cada chamada, o que pode reduzir bastante seu desempenho (GREGG, 2019). Com eBPF, o programa de coleta roda no kernel e o processo observado não é interrompido.

Três ferramentas cobrem os casos mais comuns:

```
sudo execsnoop-libbpf          # cada novo processo (execve), com
   ↪ argumentos
sudo opensnoop-libbpf          # cada arquivo aberto, por processo
sudo syscount-libbpf -i 5      # contagem de chamadas por tipo, a cada 5
   ↪  s
```

O `execsnoop` é útil para descobrir processos de vida curta, como *scripts* disparados pelo `cron`, que desaparecem antes de um `ps` ou de uma amostra de *polling* enxergá-los. O mesmo resultado pode ser obtido com uma linha de `bpftrace`, ligada ao *tracepoint* da chamada `execve`:

```
sudo bpftrace -e 'tracepoint:syscalls:sys_enter_execve {
    printf("%-6d %-16s -> %s\n", pid, comm, str(args->filename)); }'
```

Para uma visão agregada, conte as chamadas de sistema por processo. O *tracepoint* `raw_syscalls:sys_enter` dispara em qualquer chamada, de modo que o contador mostra quais processos mais usam o kernel:

```
sudo bpftrace -e 'tracepoint:raw_syscalls:sys_enter { @[comm] = count();
   ↪  }'
```

4.2 MONITORAMENTO DE CPU E MEMÓRIA

Na CPU, o método USE pergunta quanto tempo ela está ocupada (utilização) e quanto trabalho espera para rodar (saturação). A utilização está em ferramentas conhecidas, como o `top`. A saturação é o que o eBPF mostra bem: o `runqlat` mede quanto tempo cada tarefa espera na fila de execução até receber a CPU, e apresenta um histograma. O `cpudist` mostra por quanto tempo as tarefas rodam a cada vez que recebem a CPU:

```
sudo runqlat-libbpf 1 5        # histograma da fila de execucao
sudo cpudist-libbpf 1 5        # quanto tempo as tarefas rodam a cada
   ↪ vez
```

Para saber *o que* consome CPU, use amostragem: a cada 99 Hz o kernel registra o processo (e, se pedido, a pilha de chamadas) que está rodando. A taxa de 99 Hz, e não 100, evita sincronizar com eventos periódicos do sistema:

```
sudo bpftrace -e 'profile:hz:99 { @[comm] = count(); }'
sudo profile-libbpf -F 99 10   # com pilhas de chamadas, durante 10 s
```

Na memória, o eBPF não substitui `free` ou `/proc/meminfo` para o uso total, mas mostra *quem* aloca e quando. O *tracepoint* `kmem:mm_page_alloc` conta alocações de páginas por processo, o `cachestat` informa acertos e falhas no *page cache*, o `oomkill` registra processos mortos por falta de memória e o `memleak` aponta alocações que não foram liberadas em um processo:

```
sudo bpftrace -e 'tracepoint:kmem:mm_page_alloc { @[comm] = count(); }'
sudo cachestat-libbpf 1 5
sudo oomkill-libbpf
sudo memleak-libbpf -p PID
```

O `memleak` acrescenta custo ao processo observado, pois instrumenta cada alocação. Use-o por períodos curtos e fora de produção, quando possível.

4.3 MONITORAMENTO DE DISCO E I/O

No disco, utilização é a fração do tempo em que o dispositivo está ocupado, saturação é a profundidade da fila e o erro mais relevante é a latência alta. Ferramentas como o `iostat` dão médias, que escondem a cauda: um disco com latência média de 1 ms pode ter requisições de 100 ms que travam uma aplicação. O `biolatency` mostra a distribuição completa em um histograma, e o `biosnoop` lista cada requisição de bloco, com processo, tamanho e latência:

```
sudo biolatency-libbpf -D 10 1   # histograma por disco, 1 amostra de 10
   ↪ s
sudo biosnoop-libbpf             # cada requisicao de I/O
```

O mesmo histograma pode ser montado com `bpftrace`. O *script* guarda o instante em que cada requisição é emitida e, quando ela termina, calcula a diferença. A chave do mapa combina dispositivo e setor, que identificam a requisição:

```
tracepoint:block:block_rq_issue {
    @inicio[args->dev, args->sector] = nsecs;
}
tracepoint:block:block_rq_complete
/@inicio[args->dev, args->sector]/ {
    @us = hist((nsecs - @inicio[args->dev, args->sector]) / 1000);
    delete(@inicio[args->dev, args->sector]);
}
```

Salve o *script* como `biolat.bt` e execute com `sudo bpftrace biolat.bt`. A saída é um histograma em potências de dois, em microssegundos. Barras concentradas à esquerda indicam disco rápido, e uma cauda longa à direita indica pedidos esporádicos muito lentos.

4.4 MONITORAMENTO DE APLICAÇÕES

Para aplicações, o eBPF observa o que cruza o kernel, como conexões de rede, e, com sondas em espaço do usuário, o que acontece dentro do próprio programa (seção 6.4). Na rede, as ferramentas mostram o ciclo de vida das conexões TCP sem capturar pacotes:

```
sudo tcpconnect-libbpf          # conexoes abertas, por processo
sudo tcplife-libbpf -L 8080     # duracao e bytes por conexao, porta
   ↪ 8080
```

O `tcplife` responde perguntas como quais clientes mantêm conexões longas com o serviço e quantos bytes cada uma trafegou. Para gerar tráfego de teste, rode `python3 -m http.server 8080` em outro terminal e acesse a porta com o `curl`. As retransmissões TCP, um indício melhor de degradação da rede do que a média de latência, podem ser observadas com um *tracepoint*:

```
sudo bpftrace -e 'tracepoint:tcp:tcp_retransmit_skb {
    printf("%s sport=%d dport=%d\n", comm, args->sport, args->dport); }'
```

Quando o dado necessário não está nessas ferramentas, a opção é escrever uma sonda própria, como a contagem de bytes por porta da seção 5.3. Diferente da instrumentação dentro da aplicação, a coleta com eBPF não exige alterar nem reiniciar o serviço, mas só enxerga o que o kernel vê. Eventos internos, como o tempo de uma consulta SQL, precisam de sondas `uprobe` ou USDT (capítulo 6).

5 FERRAMENTAS DE OBSERVABILIDADE

Há três formas de usar o eBPF para observabilidade, em ordem crescente de esforço: rodar ferramentas prontas, escrever *scripts* curtos e escrever um programa próprio. Este capítulo apresenta o BCC, o `bpftrace` e a `libbpf`, que cobrem esses três níveis, e os *exporters*, que levam o resultado ao Prometheus. O Quadro 3 resume quando usar cada um.

Quadro 3 – Comparação entre as ferramentas de eBPF

Ferramenta	Linguagem	Melhor uso	Ponto de atenção
BCC	Python ou C, e ferramentas prontas	diagnóstico imediato com ferramentas existentes	compila com LLVM a cada execução, exige cabeçalhos do kernel
bpftrace	linguagem própria, de alto nível	investigação interativa em uma linha	não serve para programas de longa duração com lógica complexa
libbpf	C, com CO-RE	coletores contínuos e distribuíveis	maior esforço de desenvolvimento
ebpf_exporter	YAML e C	métricas para o Prometheus	exige escrever o programa C de cada métrica

Fonte: Elaborado pelos autores, a partir de IO Visor Project (s.d.), bpftrace Contributors (s.d.), libbpf Contributors (s.d.), Cloudflare (s.d.).

5.1 BCC

O BCC (*BPF Compiler Collection*) é um conjunto de ferramentas e uma biblioteca para criar programas eBPF, com interface em Python e C (IO Visor Project, s.d.). Sua maior contribuição ao dia a dia é a coleção de ferramentas prontas, como as usadas no capítulo 4 (`execsnoop`, `biolatency`, `tcplife`), que cobrem a maior parte dos diagnósticos comuns sem escrever código. No Ubuntu, instalam-se com o pacote `bpfcc-tools`, e cada ferramenta recebe o sufixo `-bpfcc`.

Para casos que as ferramentas não cobrem, o BCC permite escrever o programa eBPF em C dentro de um *script* Python. O exemplo abaixo imprime uma mensagem a cada chamada `openat`:

```
from bcc import BPF

programa = """
int hello(void *ctx) {
    bpf_trace_printk("openat\\n");
    return 0;
}
```

```
"""
b = BPF(text=programa)
b.attach_kprobe(event=b.get_syscall_fnname("openat"), fn_name="hello")
b.trace_print()
```

A limitação do BCC é que ele compila o programa em C no momento da execução, usando o LLVM e os cabeçalhos do kernel instalados na máquina. Isso significa instalar dependências pesadas em cada *host*, esperar a compilação a cada início e manter os cabeçalhos sincronizados com o kernel (NAKRYIKO, 2020). Na prática, o problema aparece com kernels recentes: no ambiente de testes deste manual (Ubuntu 26.04, kernel 7.0 e BCC 0.35.0), 11 das 14 ferramentas prontas do BCC testadas falharam com erros de compilação nos cabeçalhos do kernel, como `static assertion failed` e `no member named 'ns_id' in 'struct ns_common'`. O exemplo em Python acima funcionou porque não inclui cabeçalhos do kernel. Programas que dependem de estruturas internas, como a maioria das ferramentas prontas, quebram quando o kernel muda mais rápido que o BCC.

É o problema que o CO-RE, descrito no capítulo 1, resolve. O pacote `libbpf-tools` reúne versões CO-RE das mesmas ferramentas, com o sufixo `-libbpf`, que não compilam nada na execução. No mesmo ambiente, 13 das 14 foram executadas sem erro, e por isso este manual as prefere (capítulo 4).

5.2 BPFTRACE

O `bpftrace` é uma linguagem de alto nível para eBPF, inspirada no `awk` e no DTrace (bpftrace Contributors, s.d.). Um programa é uma lista de sondas, cada uma com um filtro opcional e uma ação:

```
sonda /filtro/ { acao }
```

Os elementos que mais aparecem são as variáveis embutidas (`pid`, `comm`, `nsecs`, `args`, `retval`), os mapas, que começam com `@`, e as funções de agregação (`count`, `sum`, `avg`, `hist`). Ao encerrar com `Ctrl+C`, o `bpftrace` imprime todos os mapas. Para listar as sondas disponíveis e os argumentos de uma delas, use a opção `-l`:

```
sudo bpftrace -l 'tracepoint:syscalls:*openat*'
sudo bpftrace -lv tracepoint:syscalls:sys_enter_openat
```

O exemplo a seguir soma os bytes enviados por cada processo e imprime o total a cada segundo. O terceiro argumento de `tcp_sendmsg`, acessível como `arg2`, é o tamanho da mensagem:

```
sudo bpftrace -e '
kprobe:tcp_sendmsg { @bytes[comm] = sum(arg2); }
interval:s:1 { print(@bytes); clear(@bytes); }'
```

O bpftrace é a melhor escolha para investigar um problema em andamento. Para um coletor que ficará dias em execução, com mapas exportados a um painel, um programa em libbpf é mais adequado.

5.3 LIBBPF

A libbpf é a biblioteca que carrega e gerencia programas eBPF: lê o objeto compilado, aplica as realocações CO-RE, envia o programa ao kernel pela chamada bpf(), anexa aos *hooks* e administra os mapas (libbpf Contributors, s.d.). Os programas escritos com ela são compilados uma única vez e distribuídos como binários, sem compilador no destino.

O exemplo a seguir conta os bytes enviados por porta de destino TCP, a mesma ideia do coletor usado no experimento do TCC (ALVES; PLINTA, 2026). O programa que roda no kernel fica em monitor.bpf.c:

```
#include "vmlinux.h"
#include <bpf/bpf_helpers.h>
#include <bpf/bpf_tracing.h>
#include <bpf/bpf_core_read.h>
#include <bpf/bpf_endian.h>

struct {
    __uint(type, BPF_MAP_TYPE_HASH);
    __uint(max_entries, 1024);
    __type(key, __u16);    /* porta TCP de destino */
    __type(value, __u64);  /* bytes acumulados */
} bytes_por_porta SEC(".maps");

SEC("kprobe/tcp_sendmsg")
int BPF_KPROBE(trace_tcp_sendmsg, struct sock *sk,
               struct msghdr *msg, size_t size)
{
    __u16 porta = bpf_ntohs(BPF_CORE_READ(sk, __sk_common.skc_dport));
    __u64 *total = bpf_map_lookup_elem(&bytes_por_porta, &porta);

    if (total) {
        __sync_fetch_and_add(total, size);
    } else {
        __u64 inicial = size;
        bpf_map_update_elem(&bytes_por_porta, &porta, &inicial,
   ↪ BPF_NOEXIST);
    }
    return 0;
}
```

```
char LICENSE[] SEC("license") = "GPL";
```

Alguns detalhes importam. A porta no *socket* está em ordem de bytes de rede, por isso a conversão com `bpf_ntohs`. A soma é feita com uma operação atômica, pois o mesmo programa pode rodar em vários núcleos ao mesmo tempo. E a macro `BPF_CORE_READ` é o que permite a leitura do campo de forma portável entre versões do kernel.

A compilação usa o `clang` com alvo `bpf`, e o `vmlinux.h` gerado no capítulo 2:

```
clang -O2 -g -target bpf -D__TARGET_ARCH_x86 \
      -I . -c monitor.bpf.c -o monitor.bpf.o
```

Dependendo do kernel, o `clang` imprime avisos do tipo `declaration does not declare anything` vindos do `vmlinux.h`. São inofensivos, e o objeto `monitor.bpf.o` é gerado normalmente. Em máquinas ARM64, troque `__TARGET_ARCH_x86` por `__TARGET_ARCH_arm64`.

Para carregar o objeto e ler o mapa sem escrever um programa de controle em C, o `bpftool` faz as duas tarefas:

```
sudo bpftool prog loadall monitor.bpf.o /sys/fs/bpf/monitor autoattach
sudo bpftool map dump name bytes_por_porta
```

Depois de algumas requisições a um servidor local na porta 8080 (por exemplo, `python3 -m http.server 8080` e alguns `curl`), o mapa mostra os bytes acumulados por porta de destino. Este é um trecho da saída:

```
[{
        "key": 8080,
        "value": 390
    },{
        "key": 35098,
        "value": 2694
    },
    ...
]
```

A chave 8080 é a porta do servidor de teste. As demais são portas efêmeras de clientes, para as quais o *host* também enviou dados. O programa fica carregado enquanto o diretório fixado existir. Para descarregá-lo, remova-o com `sudo rm -r /sys/fs/bpf/monitor`.

Em um coletor real, o programa de controle em espaço do usuário usa a API da `libbpf` para abrir o objeto, carregá-lo, anexar os programas e ler os mapas periodicamente. O `bpftool gen skeleton` gera um cabeçalho em C que automatiza esses passos.

5.4 EBPF EXPORTERS

Um *exporter* expõe métricas em um *endpoint* HTTP para o Prometheus buscar (TURNBULL, 2018). O `ebpf_exporter`, da Cloudflare, faz isso para métricas coletadas com eBPF: o programa em C, escrito com `libbpf`, preenche mapas no kernel, e um arquivo YAML descreve como cada mapa vira uma métrica do Prometheus (Cloudflare, s.d.). Em termos de uso, funciona como as ferramentas do BCC, mas com a saída em formato de métricas.

A configuração a seguir, do próprio repositório do projeto, define um contador chamado `timer_starts_total`, que conta os temporizadores do kernel disparados, com um rótulo `function` decodificado a partir do endereço da função:

```
metrics:
  counters:
    - name: timer_starts_total
      help: Timers fired in the kernel
      labels:
        - name: function
          size: 8
          decoders:
            - name: ksym
```

O projeto publica, em suas *releases*, um pacote com o binário e os exemplos já compilados (`ebpf_exporter_with_examples`), o que dispensa compilar o programa. Depois de extraído, o *exporter* é executado como processo com privilégios, apontando para a pasta com as configurações, e passa a expor as métricas na porta 9435:

```
sudo ./ebpf_exporter --config.dir=examples --config.names=timers
```

Em um teste com a versão 2.5.1, o *exporter* iniciou em cerca de 0,3 s e passou a responder em `http://localhost:9435/metrics` com linhas como:

```
ebpf_exporter_timer_starts_total{function="blk_stat_timer_fn"} 42
ebpf_exporter_timer_starts_total{function="commit_timeout"} 27
```

O repositório inclui exemplos prontos, como o histograma de latência de disco (`biolatency`), e também suporta o envio de *traces* no formato OpenTelemetry (Cloudflare, s.d.). A ligação com o Prometheus e o Grafana é o assunto do capítulo 7.

6 INSTRUMENTAÇÃO COM EBPF

Instrumentar é escolher em que ponto do sistema o programa eBPF vai observar. Essa escolha define o que se enxerga, a estabilidade do código ao longo das versões do kernel e o custo da coleta. Este capítulo apresenta os seis mecanismos mais usados em observabilidade: *kprobes*, *kretprobes*, *tracepoints*, *uprobes*, USDT e *perf events*. O Quadro 4 os compara.

Quadro 4 – Mecanismos de instrumentação do eBPF

Mecanismo	**Onde observa**	**Estabilidade**	**Quando usar**
kprobe	entrada de qualquer função do kernel	baixa: nomes e argumentos mudam entre versões	não há *tracepoint* para o que se quer
kretprobe	saída de uma função do kernel	baixa	medir retorno e latência de funções
tracepoint	pontos fixos definidos pelo kernel	alta	primeira escolha no kernel
uprobe	função de um binário em espaço do usuário	depende do binário	observar aplicações sem alterá-las
USDT	pontos definidos pelo desenvolvedor da aplicação	alta, se o programa os mantiver	aplicações que já trazem sondas
perf event	eventos de hardware e software, por amostragem	alta	perfil de CPU

Fonte: Elaborado pelos autores, a partir de Gregg (2019), Linux Kernel Documentation (2024).

6.1 KPROBES

Um *kprobe* permite anexar um programa à entrada de praticamente qualquer função do kernel, sem que o kernel precise ter sido preparado para isso (Linux Kernel Documentation, 2024). É o mecanismo mais flexível e o usado no coletor de bytes por porta do capítulo 5, anexado a `tcp_sendmsg`. Os argumentos da função ficam disponíveis ao programa, e no `bpftrace` aparecem como `arg0`, `arg1` e assim por diante.

A flexibilidade tem um preço: as funções internas do kernel não são uma interface estável. Uma função pode mudar de nome, de argumentos ou deixar de existir em uma versão nova, e o programa passa a falhar ou, pior, a ler dados errados. Por isso o *kprobe* deve ser a escolha quando não existe um *tracepoint* equivalente. Para ver quais funções podem ser instrumentadas:

```
sudo bpftrace -l 'kprobe:tcp_*'
sudo bpftrace -e 'kprobe:tcp_sendmsg { @bytes[comm] = sum(arg2); }'
```

6.2 KRETPROBES

O *kretprobe* é o par do *kprobe*: dispara quando a função retorna, e dá acesso ao valor de retorno. O uso típico é medir a duração de uma função, combinando os dois. O *kprobe* guarda o instante de entrada em um mapa, indexado pela *thread*, e o *kretprobe* calcula a diferença:

```
sudo bpftrace -e '
kprobe:vfs_read { @inicio[tid] = nsecs; }
kretprobe:vfs_read /@inicio[tid]/ {
    @us = hist((nsecs - @inicio[tid]) / 1000);
    delete(@inicio[tid]);
}'
```

O valor de retorno também é útil por si só. Em `vfs_read`, ele é o número de bytes lidos, e `hist(retval)` mostra a distribuição dos tamanhos de leitura. O *kretprobe* costuma custar mais do que o *kprobe*, porque o kernel precisa interceptar o retorno da função. Em kernels recentes (5.5 ou superior), os programas do tipo `fentry` e `fexit` fazem o mesmo com menos custo, e são preferíveis quando disponíveis.

6.3 TRACEPOINTS

Tracepoints são pontos de instrumentação definidos de forma explícita pelos desenvolvedores do kernel, em locais estratégicos como chamadas de sistema, escalonador e camada de blocos. Por serem mantidos de propósito, mudam muito menos entre versões do que as funções internas, e já trazem os argumentos descritos (Linux Kernel Documentation, 2024). Por isso são a primeira opção quando existem.

A lista de *tracepoints* está em `/sys/kernel/tracing/events`, e o `bpftrace` mostra os campos de cada um:

```
sudo bpftrace -l 'tracepoint:sched:*'
sudo bpftrace -lv tracepoint:block:block_rq_issue
sudo bpftrace -e 'tracepoint:sched:sched_process_exec {
    printf("%s\n", str(args->filename)); }'
```

Os exemplos do capítulo 4 sobre chamadas de sistema (`syscalls:sys_enter_openat`) e disco (`block:block_rq_issue`) usam *tracepoints*. A limitação é a cobertura: só existe um *tracepoint* onde alguém decidiu colocar um.

6.4 UPROBES

O *uprobe* é o equivalente do *kprobe* para o espaço do usuário: anexa o programa a uma função de um binário ou biblioteca, identificada pelo caminho do arquivo e pelo nome do símbolo. Permite observar uma aplicação sem alterá-la nem reiniciá-la (GREGG, 2019). Um exemplo clássico é o *shell* `bash`, cuja função `readline` devolve cada linha digitada:

```
sudo bpftrace -e 'uretprobe:/bin/bash:readline { printf("%s\n", str(
   ↪ retval)); }'
```

A dependência do binário é a limitação principal. O *uprobe* precisa de um símbolo visível: binários sem a tabela de símbolos (*stripped*) ou com nomes alterados por otimização podem não ser instrumentáveis, e uma atualização do programa pode renomear a função. Para listar os símbolos de um binário, use `bpftrace -l 'uprobe:/caminho/do/binario:*'`.

6.5 USDT

USDT (*User Statically-Defined Tracing*) são sondas que o próprio desenvolvedor da aplicação inseriu no código, de forma estática, em pontos significativos: início de uma consulta, nova conexão, coleta de lixo. Como são mantidas pelo projeto, têm a estabilidade dos *tracepoints* e o significado de eventos de negócio, algo que um *uprobe* em funções internas não oferece.

Servidores de banco de dados e ambientes de execução, como PostgreSQL, MySQL, a JVM e o Python, podem trazer sondas USDT, dependendo de como foram compilados. Um exemplo que está à mão em muitas distribuições é o interpretador Python. Para ver as sondas de um binário, use a opção `-l` do `bpftrace`:

```
sudo bpftrace -l 'usdt:/usr/bin/python3.14:*'
```

No Ubuntu 26.04, o `python3.14` expõe cinco sondas:

- `python:audit`;
- `python:gc__start` e `python:gc__done`, no início e no fim da coleta de lixo;
- `python:import__find__load__start` e `python:import__find__load__done`, na importação de módulos.

A lista depende da compilação: a sonda `python:function__entry`, que aparece na documentação do Python, não existe nesse pacote, e o `bpftrace` acusa `No matches for usdt`. Por isso é sempre necessário listar as sondas antes de escrever o programa.

A sonda `import__find__load__start` recebe o nome do módulo no primeiro argumento. O comando a seguir imprime cada módulo importado pelos programas Python do sistema, o que ajuda, por exemplo, a descobrir por que um *script* demora a iniciar:

```
sudo bpftrace -e 'usdt:/usr/bin/python3.14:python:
   ↪ import__find__load__start {
    printf("%s\n", str(arg0)); }'
```

Em um teste, a saída listou os módulos na ordem em que o interpretador os carregou (`site`, `os`, `encodings` e outros). No ambiente de teste (Ubuntu 26.04, kernel 7.0 e `bpftrace` 0.25.0), o `bpftrace` imprimiu, antes da saída, avisos de compilação vindos do `vmlinux.h` (`declaration does not declare anything`). São inofensivos, e a opção `-no-warnings` não os suprime. Além disso, um programa USDT de várias linhas, salvo em arquivo, fez o `bpftrace` abortar com `Assertion failed`, enquanto o mesmo programa escrito em uma única linha com `-e` funcionou. Nessa combinação de versões, prefira *scripts* USDT de uma linha.

Se o programa não traz sondas, a alternativa é o *uprobe*, com a estabilidade menor descrita acima.

6.6 PERF EVENTS

Os *perf events* são a infraestrutura de contadores e amostragem do kernel, a mesma usada pelo `perf`. O eBPF pode anexar programas a eles para amostrar o que o sistema faz em intervalos regulares, em vez de reagir a cada evento. Isso torna o custo previsível, independente de quantos eventos ocorrem, e é o mecanismo por trás dos perfis de CPU e dos gráficos de chamas (GREGG, 2019).

O `bpftrace` expõe os *perf events* de software e hardware. Contadores de hardware, como falhas de cache, dependem de suporte do processador e geralmente não estão disponíveis dentro de máquinas virtuais:

```
sudo bpftrace -e 'profile:hz:99 { @[kstack] = count(); }'
sudo bpftrace -e 'software:page-faults:1 { @[comm] = count(); }'
```

O primeiro comando amostra a pilha do kernel 99 vezes por segundo em cada CPU, o que é suficiente para identificar onde o tempo é gasto com custo baixo. O segundo conta as falhas de página por processo.

7 COLETA E VISUALIZAÇÃO DE DADOS

Uma métrica coletada no kernel só ajuda quando alguém consegue vê-la ao longo do tempo. Este capítulo mostra como levar os dados do eBPF até o Prometheus, que os armazena, e ao Grafana, que os exibe em painéis. O caminho é: programa eBPF, mapa, *exporter*, Prometheus e Grafana.

7.1 EBPF E PROMETHEUS

O Prometheus segue o modelo de *pull*: um servidor busca, em intervalos regulares, as métricas expostas em um *endpoint* HTTP e as armazena como séries temporais (Prometheus Authors, s.d.; TURNBULL, 2018). Para um coletor eBPF, isso significa que algum processo precisa ler os mapas do kernel e expô-los nesse formato. Há três caminhos:

- usar o `ebpf_exporter`, que faz isso a partir de um arquivo de configuração (seção 5.4);
- escrever um *exporter* próprio, em Python, que lê os mapas (seção 7.3);
- gravar as métricas em um arquivo de texto e deixar o *textfile collector* do Node Exporter publicá-las, útil para coletas esporádicas.

Com um *exporter* rodando na porta 9435, a configuração mínima do Prometheus para buscá-lo é:

```
scrape_configs:
  - job_name: ebpf
    scrape_interval: 5s
    static_configs:
      - targets: ['localhost:9435']
```

Em um teste com o Prometheus 3.15.0 e o `ebpf_exporter` 2.5.1, o alvo apareceu como `up` em *Status* > *Targets* e as métricas ficaram disponíveis para consulta. O intervalo de coleta (`scrape_interval`) define a resolução do que o Prometheus enxerga. A agregação no kernel continua contando cada evento, mas o Prometheus só vê o valor acumulado a cada intervalo. Para contadores, isso não causa perda: a diferença entre duas leituras é exatamente o que aconteceu no período. Para valores instantâneos, como uma latência, o ideal é que o programa eBPF já entregue um histograma.

7.2 EBPF E GRAFANA

O Grafana é uma ferramenta de visualização que consulta fontes de dados, entre elas o Prometheus, e as apresenta em painéis (Grafana Labs, s.d.). O uso básico é cadastrar o Prometheus como fonte de dados e criar painéis com consultas na linguagem PromQL.

Os tipos de painel mais úteis para métricas do eBPF são o gráfico de séries temporais, para taxas, e o mapa de calor (*heatmap*), para distribuições de latência. Duas consultas de exemplo, usando as métricas de exemplo do `ebpf_exporter`:

```
# taxa de temporizadores disparados no kernel por segundo
rate(ebpf_exporter_timer_starts_total[1m])

# latencia de disco no percentil 95
histogram_quantile(0.95,
  sum by (le) (rate(ebpf_exporter_bio_latency_seconds_bucket[5m])))
```

No teste com o Grafana 13.2.3, o Prometheus foi cadastrado como fonte de dados (URL `http://localhost:9090`), e as duas consultas acima retornaram resultados pela fonte de dados: a primeira trouxe 24 séries, e a segunda, uma latência no percentil 95 de cerca de 5,6 ms, medida depois de gravar 300 MB em disco com `dd`. A criação dos painéis em si (séries temporais e mapa de calor) não foi exercitada nesse teste.

A função `rate` converte o contador acumulado em uma taxa por segundo. Já `histogram_quantile` calcula percentis a partir dos *buckets* do histograma, e é por isso que a latência de disco deve ser exportada como histograma, e não como média.

7.3 EXPORTAÇÃO DE MÉTRICAS

Ao transformar um mapa eBPF em métrica, a escolha do tipo segue o que o mapa guarda. Valores que só crescem, como bytes enviados, viram contadores. Valores que sobem e descem, como conexões abertas, viram *gauges*. Distribuições, como latência, viram histogramas (TURNBULL, 2018). Algumas convenções evitam confusão: nomes de contadores terminam em `_total`, e unidades de tempo são expressas em segundos.

O cuidado mais importante é a cardinalidade dos rótulos. Cada combinação de valores de rótulo cria uma série temporal nova. Usar o PID ou o endereço IP do cliente como rótulo pode gerar milhares de séries e sobrecarregar o Prometheus. Prefira rótulos de poucos valores, como a porta de serviço, e agregue o resto.

O exemplo a seguir é um *exporter* em Python que lê o mapa `bytes_por_porta` do programa do capítulo 5, já carregado no kernel, e o publica na porta 8000. A leitura usa o `bpftool`, que devolve o conteúdo do mapa em JSON, e a classe `Coletor` faz essa consulta a cada requi-

sição do Prometheus:

```
import json
import subprocess
import time

from prometheus_client import start_http_server
from prometheus_client.core import CounterMetricFamily, REGISTRY

PORTAS = {80, 443, 8080}  # portas de servico de interesse


def ler_mapa(nome):
    saida = subprocess.run(["bpftool", "-j", "map", "dump", "name", nome
    ↪ ],
                           capture_output=True, text=True, check=True).
    ↪ stdout
    return json.loads(saida)


class Coletor:
    def collect(self):
        m = CounterMetricFamily("ebpf_tcp_bytes_enviados",
                                "Bytes enviados por porta de destino",
                                labels=["porta"])
        for entrada in ler_mapa("bytes_por_porta"):
            f = entrada["formatted"]
            if f["key"] in PORTAS:
                m.add_metric([str(f["key"])], f["value"])
        yield m


REGISTRY.register(Coletor())
start_http_server(8000)
while True:
    time.sleep(1)
```

O filtro `PORTAS` não é um detalhe. No teste deste manual, sem o filtro, o *exporter* publicou 21 séries depois de apenas cinco requisições, quase todas de portas efêmeras de clientes (como 58674 ou 35098), cada uma com poucos bytes. Em um servidor real, esse número cresce a cada nova conexão. Com o filtro, o mesmo teste publicou apenas três séries, uma para cada porta listada. Para o servidor de teste na porta 8080, a série tem este formato:

```
# HELP ebpf_tcp_bytes_enviados_total Bytes enviados por porta de destino
# TYPE ebpf_tcp_bytes_enviados_total counter
ebpf_tcp_bytes_enviados_total{porta="8080"} 390.0
```

Por não depender de compilação em tempo de execução, esse *exporter* funciona em

qualquer kernel que carregue o programa `libbpf`. O pacote `python3-prometheus-client` traz a biblioteca. O *exporter* precisa rodar como `root`, porque o `bpftool` exige esse privilégio para ler mapas.

Esse padrão, em que o mapa acumula no kernel e o *exporter* só o lê sob demanda, evita o custo do *polling* descrito no capítulo 1: o espaço do usuário é acionado uma vez por intervalo de coleta, e não a cada evento.

7.4 COLETA DE EVENTOS

Métricas resumem eventos. Quando é preciso o evento em si, como cada execução de um processo ou cada conexão recusada, o programa eBPF envia o registro ao espaço do usuário em vez de atualizar um contador. O mecanismo para isso é o *ring buffer*, uma fila compartilhada entre o kernel e o espaço do usuário disponível a partir do kernel 5.8, que substitui o antigo *perf buffer* com menos cópias e ordenação preservada (Linux Kernel Documentation, 2024).

O fluxo é o mesmo das métricas, com uma diferença no destino. Em vez do Prometheus, que guarda números, os eventos costumam ir para um sistema de *logs* ou de análise, normalmente em formato JSON, para serem pesquisados depois. O `ebpf_exporter` também permite enviar *traces* no formato OpenTelemetry (Cloudflare, s.d.). A regra do capítulo 3, de agregar no kernel quando possível, continua valendo: registrar cada evento custa mais do que contá-lo, e deve ser reservado a eventos raros ou de alto valor, como falhas e conexões suspeitas.

7.5 DASHBOARDS DE OBSERVABILIDADE

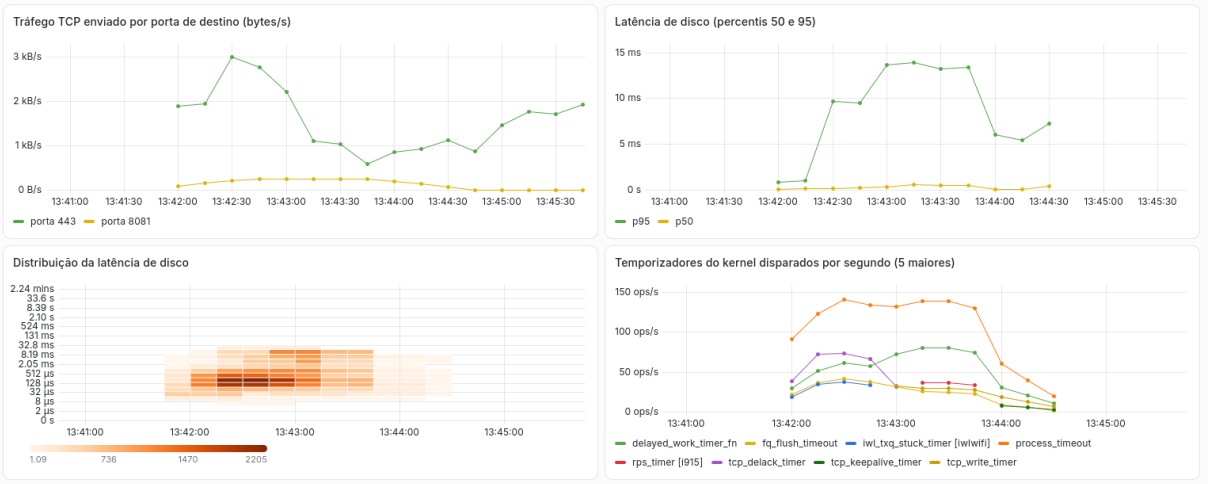
Um painel útil responde a perguntas, não exibe todas as métricas disponíveis. Um bom ponto de partida é organizá-lo pelos sinais do capítulo 3: latência, tráfego, erros e saturação (BEYER *et al.*, 2016). Para o monitoramento de um serviço com eBPF, uma organização possível é:

- **tráfego**: bytes enviados por porta de serviço e conexões por segundo (`tcpconnect`);
- **latência**: percentis de latência de disco e de resposta do serviço, a partir de histogramas;
- **erros**: retransmissões TCP e processos encerrados por falta de memória;
- **saturação**: latência da fila de execução da CPU e profundidade da fila de disco.

A Figura 1 mostra um painel montado com as métricas dos capítulos anteriores. O gráfico de tráfego usa o programa da seção 5.3 e o *exporter* em Python da seção 7.3, e os demais usam o `ebpf_exporter`: a latência de disco (percentis 50 e 95), o mapa de calor da distribuição

dessa latência e os cinco temporizadores do kernel mais frequentes. O painel foi montado a partir de quatro consultas PromQL, uma por quadro, e atualizado a cada 5 segundos.

Figura 1 – Painel do Grafana com métricas coletadas por eBPF



Fonte: Elaborado pelos autores.

Alguns cuidados ajudam a manter o painel confiável. Use intervalos de `rate` de pelo menos quatro vezes o intervalo de coleta, para que cada janela contenha várias amostras. Prefira percentis a médias para latência. Acompanhe o próprio coletor: o consumo de CPU e de memória do *exporter* deve estar no painel, porque o custo da observação é parte do que se quer controlar. E defina alertas só para sintomas que exijam ação, deixando o restante apenas para investigação.

8 EXEMPLOS PRÁTICOS

Este capítulo reúne três cenários que combinam as ferramentas dos capítulos anteriores. Cada um parte de uma pergunta de diagnóstico e usa só comandos já apresentados, de modo que o leitor possa reproduzi-los em um terminal com dois painéis: um para a ferramenta de coleta e outro para gerar a atividade observada. Todos exigem `root` e foram testados no ambiente descrito no capítulo 4.

8.1 MONITORAMENTO DE PROCESSOS

Pergunta: que processos de vida curta estão sendo criados no sistema? Um *script* que roda por milissegundos, disparado pelo `cron` ou por um serviço, não aparece em um `ps` nem em uma amostra de *polling*.

No primeiro terminal, inicie o rastreamento de `execve` e deixe-o rodando:

```
sudo execsnoop-libbpf
```

No segundo terminal, crie alguns processos curtos:

```
for i in 1 2 3; do date > /dev/null; sleep 0.2; done
```

O primeiro terminal passa a listar cada processo criado, com o PID, o processo pai e os argumentos. Processos que duram milissegundos, como esses, passam despercebidos pelo `ps`. Nos testes, o `execsnoop` listou os `sleep` e `date` do laço, mas não necessariamente todos eles: em uma máquina ocupada, parte dos eventos pode ficar de fora da saída. Para saber *quem* está criando processos em maior volume, em vez de listar cada um, troque a ferramenta por uma contagem:

```
sudo bpftrace -e 'tracepoint:syscalls:sys_enter_execve { @[comm] = count
    ↪ (); }'
```

Ao encerrar com `Ctrl+C`, o `bpftrace` imprime o mapa com o total de execuções por nome do processo que chamou `execve`, incluindo outros programas ativos na máquina (no teste apareceram o `code` e o `bash`). Um número alto em um *script* de verificação periódica é um indício de que ele cria processos demais para o que faz.

8.2 ANÁLISE DE CHAMADAS DE SISTEMA

Pergunta: o que um programa está pedindo ao kernel? Aqui o alvo é o servidor HTTP embutido do Python. Inicie-o no primeiro terminal. Os exemplos usam a porta 8099 porque

a 8080 costuma estar ocupada (no ambiente de teste, ela pertencia a um `docker-proxy`, e o servidor encerrava sem avisar). Confira com `ss -ltnp` se a porta escolhida está livre:

```
python3 -m http.server 8099
```

No segundo terminal, descubra o PID do servidor e observe os arquivos que ele abre enquanto atende requisições:

```
PID=$(pgrep -f "http.server 8099")
sudo opensnoop-libbpf -p $PID
```

Em um terceiro terminal, faça requisições com `curl http://localhost:8099/`. O `opensnoop` pode mostrar o servidor abrindo o diretório pedido (em uma das duas rodadas de teste apareceu `/tmp/`; na outra, a saída ficou vazia, então não conte com ele para ver todas as aberturas). Repare na coluna `COMM`: ela traz o nome da *thread* que atendeu a requisição (por exemplo, `Thread-3 (proce)`, e não `python3`. Por isso filtrar por nome (`-n python3`) não encontra nada, e o filtro por PID (`-p`) é o correto.

Para a visão agregada, o `syscount` mostra quais chamadas são mais feitas, sem interromper o processo, o que o `strace` não consegue evitar:

```
sudo syscount-libbpf -i 5
```

Sem filtro, a saída cobre o sistema inteiro (no teste, `futex`, `ioctl`, `ppoll` e `write` lideraram) e serve para ver o perfil geral do host. A opção `-p PID` existe, mas nos testes ela contou apenas a *thread* principal do servidor, que só aparecia em `poll`, e deixou de fora as *threads* que atendem as requisições. Para programas com várias *threads*, prefira o `bpftrace`, que permite filtrar por `pid`, o identificador do processo inteiro. No teste, ele contou as chamadas da *thread* principal e de cada *thread* de atendimento (cerca de 200 por *thread* em três requisições):

```
sudo bpftrace -e 'tracepoint:raw_syscalls:sys_enter /pid == '$PID'/ { @[
    ↪ comm] = count(); }'
```

8.3 RASTREAMENTO DE UMA APLICAÇÃO

Pergunta: quem se conecta ao serviço, por quanto tempo e com quantos bytes? Com o mesmo servidor na porta 8099, rode o `tcplife`, que registra uma linha ao fim de cada conexão:

```
sudo tcplife-libbpf -L 8099
```

Gere conexões em outro terminal:

```
for i in $(seq 1 5); do curl -s http://localhost:8099/ > /dev/null; done
```

Cada linha traz o processo, os endereços e portas, os bytes enviados e recebidos e a duração. Conexões longas ou com muitos bytes se destacam do conjunto. Para ver também quem

abre conexões a partir da máquina, e não só as recebidas, use o `tcpconnect-libbpf` em paralelo.

Esses comandos observam o serviço de fora, pelo que o kernel vê. Para olhar por dentro de um processo Python, sem alterá-lo, use a sonda USDT do interpretador (seção 6.4 e capítulo 6). O comando a seguir lista os módulos importados por qualquer programa Python, e ajuda a explicar um início lento:

```
sudo bpftrace -e 'usdt:/usr/bin/python3.14:python:
   ↪ import__find__load__start {
    printf("%s\n", str(arg0)); }'
```

Em outro terminal, execute `python3 -c "import json, csv"` e observe os módulos aparecerem na ordem em que o interpretador os carrega. Para transformar uma dessas medidas em uma série contínua em um painel, siga o caminho do capítulo 7.

9 TROUBLESHOOTING

Este capítulo reúne os problemas mais prováveis ao seguir o manual e os limites da técnica. Os erros descritos foram observados nos testes em Ubuntu 26.04 com kernel 7.0.

9.1 IMPACTO DE PERFORMANCE

Um programa eBPF roda toda vez que o evento ao qual está anexado ocorre, então o custo depende da frequência do evento e do trabalho feito a cada execução. Algumas regras práticas:

- **Prefira eventos raros.** Um *tracepoint* em `execve` dispara algumas vezes por segundo. Um em `raw_syscalls:sys_enter` dispara a cada chamada de sistema do host inteiro, o que pode chegar a milhões por segundo.
- **Agregue no kernel.** Contar com `count()` ou `hist()` custa muito menos do que imprimir cada evento com `printf`, porque o espaço do usuário só lê o resultado ao final.
- **Prefira amostragem a rastrear tudo.** Para saber onde a CPU é gasta, `profile:hz:99` tem custo fixo, independente da carga.
- **Cuidado com ferramentas pesadas.** O `memleak` instrumenta cada alocação do processo, e o `kretprobe` custa mais que o `kprobe`. Use-os por períodos curtos e, se possível, fora de produção.

Para medir o custo de uma sonda, rode a carga de trabalho com e sem ela e compare a latência ou a vazão. O resultado do experimento citado no capítulo 1 vale para o coletor testado, não para qualquer programa.

9.2 LIMITAÇÕES

Permissões. Carregar programas eBPF exige `root` (ou as capacidades `CAP_BPF` e `CAP_PERFMON`). Sem isso, as ferramentas falham com erro de permissão. Em contêineres, o acesso costuma ser bloqueado por padrão.

BCC e cabeçalhos do kernel. O BCC compila o programa na hora da execução, a partir dos cabeçalhos do kernel em uso. No kernel 7.0 do ambiente de teste, 11 das 14 ferramentas testadas do BCC falharam ao compilar, com erros como `static assertion failed` em `include/linux/fs.h`. As versões `-libbpf`, que usam CO-RE, funcionaram em 13 das 14. Se uma ferramenta do BCC não compilar, procure a equivalente com o sufixo `-libbpf`.

Argumentos diferentes entre versões. A mesma ferramenta pode aceitar argumentos distintos na versão BCC e na `libbpf`. O `ext4slower-libbpf 10`, por exemplo, falhou com `Too many arguments`. Consulte sempre `-help` da versão instalada.

Avisos do bpftrace. Ao usar USDT, o `bpftrace` 0.25.0 imprime avisos de compilação vindos do `vmlinux.h` (`declaration does not declare anything`). São inofensivos e a opção `-no-warnings` não os suprime. Já um programa USDT de várias linhas, salvo em arquivo, fez o `bpftrace` abortar com `Assertion failed`. Escreva *scripts* USDT em uma única linha com `-e`.

Filtros por nome e por PID em programas com *threads*. O campo `COMM` traz o nome da *thread*, que pode ser diferente do nome do programa: o servidor do Python apareceu como `Thread-3 (proce`, e `opensnoop-libbpf -n python3` não mostrou nada. O `syscount-libbpf -p PID` contou só a *thread* principal. Filtre por PID no `opensnoop` e, para o `syscount`, use o `bpftrace` com `/pid == N/`.

Porta já em uso. Se o serviço de teste encerra sem erro visível, a porta provavelmente está ocupada, como a 8080 pelo `docker-proxy` no ambiente de teste. Confira com `ss -ltnp` e redirecione a saída do servidor para um arquivo para ver a mensagem de erro.

Sondas que não existem. Uma sonda pode estar ausente porque o pacote foi compilado sem ela. O erro típico é `No matches for usdt`. Liste as sondas com `bpftrace -l` antes de escrever o programa.

Funções internas mudam. Um *kprobe* depende do nome e dos argumentos de uma função interna do kernel, que não são uma interface estável. Se um programa deixar de funcionar após uma atualização, verifique primeiro se a função ainda existe com `bpftrace -l 'kprobe:nome*'`.

Contadores de hardware. Eventos como falhas de cache dependem do processador e geralmente não estão disponíveis em máquinas virtuais.

O que o eBPF não vê. A coleta enxerga o que passa pelo kernel. Eventos internos de uma aplicação, como o tempo de uma consulta SQL, exigem sondas `uprobe` ou USDT, ou instrumentação no próprio código.

10 CONCLUSÃO

Este manual percorreu o caminho de uma observação ao primeiro painel de observabilidade com eBPF. Partiu dos conceitos (capítulos 1 e 4), passou pelas ferramentas e pelos mecanismos de instrumentação (capítulos 5 e 6) e terminou na exportação de métricas para o Prometheus e na visualização no Grafana (capítulo 7). Os exemplos práticos do capítulo 8 mostram que as ferramentas prontas, como `execsnoop`, `opensnoop`, `syscount` e `tcplife`, respondem à maior parte das perguntas de diagnóstico sem escrever código.

O eBPF não substitui as ferramentas tradicionais. Ele complementa o `top`, o `iostat` e o Prometheus onde eles são mais fracos: eventos de curta duração, distribuições de latência em vez de médias e métricas por processo ou por conexão, coletadas sem alterar a aplicação. Em troca, exige `root`, um kernel recente e atenção ao custo de cada sonda, como o capítulo 9 detalha.

Os testes que sustentam os comandos deste manual estão na pasta `testes/` do repositório, com a saída de cada execução. Como os resultados dependem do kernel e das versões das ferramentas, o leitor deve repeti-los em seu próprio ambiente antes de tomar qualquer número como referência. Como próximos passos, os Manuais 2 e 3 desta série aplicam o mesmo conjunto de técnicas a segurança e a redes.

REFERÊNCIAS

ALVES, R. C.; PLINTA, J. P. S. H. **Monitoramento de Funções Virtualizadas de Redes: Um Estudo Comparativo entre Ferramentas de Extração de Comportamento**. Pinhais: Instituto Federal do Paraná, 2026. Trabalho de Conclusão de Curso (Bacharelado em Ciência da Computação).

BEYER, B. *et al.* **Site Reliability Engineering: How Google Runs Production Systems**. Sebastopol: O'Reilly Media, 2016.

bpftrace Contributors. **bpftrace: High-level tracing language for Linux**. s.d. Disponível em: <https://github.com/bpftrace/bpftrace>. Acesso em: 6 out. 2026.

CASSAGNES, C. *et al.* The rise of eBPF for non-intrusive performance monitoring. In: **NOMS 2020 – 2020 IEEE/IFIP Network Operations and Management Symposium**. [S.l.]: IEEE Press, 2020. p. 1–7. Disponível em: <https://doi.org/10.1109/NOMS47738.2020.9110434>.

Cloudflare. **ebpf_exporter: Prometheus exporter for custom eBPF metrics and OpenTelemetry traces**. s.d. Disponível em: <https://github.com/cloudflare/ebpf_exporter>. Acesso em: 6 out. 2026.

Grafana Labs. **Grafana documentation**. s.d. Disponível em: <https://grafana.com/docs/grafana/latest/>. Acesso em: 6 out. 2026.

GREGG, B. **BPF Performance Tools**. Boston: Addison-Wesley, 2019.

____. **Systems Performance: Enterprise and the Cloud**. 2. ed. Boston: Addison-Wesley, 2020.

IO Visor Project. **BCC: Tools for BPF-based Linux IO analysis, networking, monitoring, and more**. s.d. Disponível em: <https://github.com/iovisor/bcc>. Acesso em: 6 out. 2026.

libbpf Contributors. **libbpf: Automated upstream mirror for libbpf stand-alone build**. s.d. Disponível em: <https://github.com/libbpf/libbpf>. Acesso em: 6 out. 2026.

Linux Kernel Documentation. **BPF Documentation**. 2024. Disponível em: <https://www.kernel.org/doc/html/latest/bpf/>.

MAJORS, C.; FONG-JONES, L.; MIRANDA, G. **Observability Engineering: Achieving Production Excellence**. Sebastopol: O'Reilly Media, 2022.

MCCANNE, S.; JACOBSON, V. The BSD packet filter: A new architecture for user-level packet capture. In: **Proceedings of the USENIX Winter 1993 Conference**. San Diego: [s.n.], 1993. p. 259–270.

NAKRYIKO, A. **BPF CO-RE (Compile Once – Run Everywhere)**. 2020. Disponível em: <https://nakryiko.com/posts/bpf-portability-and-co-re/>.

Prometheus Authors. **Prometheus: Overview**. s.d. Disponível em: <https://prometheus.io/docs/introduction/overview/>. Acesso em: 6 out. 2026.

RESEARCH, D. P. **Passive TCP RTT Latency Monitoring and Aggregation in the Linux Kernel using epping and eBPF**. [S.l.], 2024.

TURNBULL, J. **Monitoring with Prometheus**. [S.l.]: Turnbull Press, 2018.